

# Engine Specification: Premium Automaton Micro-Funnel  
Code-name: Match Engine 2.0 (Asynchronous Build)

## ## 1. Structural Polish & Advanced Mechanics

### ### 1.1 Evasive "No" Button Proximity Physics

The "No" choice interaction operates as a playful, un-clickable element. Using vector tracking formulas, the selection target dynamically flees the user's cursor or touch point whenever an intersection threshold is breached. To keep the interaction fluid and organic, the interface applies standard Newtonian spring physics rather than immediate position snapping.

Let  $P_{\text{cursor}} = (x_c, y_c)$  represent the live coordinates of the tracking vector point. Let  $P_{\text{button}} = (x_b, y_b)$  establish the current center coordinates of the element layout box.

The system constantly computes the Euclidean spatial proximity delta vector:

$$\vec{D} = P_{\text{button}} - P_{\text{cursor}}$$

$$d = \sqrt{(x_b - x_c)^2 + (y_b - y_c)^2}$$

When the distance evaluation matches  $d < r_{\text{crit}}$  (where the critical activation boundary radius  $r_{\text{crit}}$  is set natively to 75 pixels), the engine calculates a new target position by extending the normalized heading vector with an explosive velocity multiplier scale factor ( $V_{\text{scale}}$ ):

$$\hat{u} = \frac{\vec{D}}{d}$$

$$P_{\text{new}} = P_{\text{button}} + \hat{u} \cdot V_{\text{scale}}$$

To ensure perfect preservation of cross-device viewports, if  $P_{\text{new}}$  maps outside the boundary constraints of the client window, the trajectory instantly undergoes a hard inversion bounce calculation against the edge matrix boundary, reflecting the button back into open visual screen real estate. The displacement rendering applies a Framer Motion spring profile: `{ mass: 0.5, tension: 210, friction: 14 }`.

### ### 1.2 AI Aesthetic Match Scripting Engine

Upon final submission of lead identity variables, user profile selections are parsed through a structured client script against 120 distinct pre-configured scenario variations. Rather than running slow network calls mid-interaction, this client matrix instantly builds a highly customized agenda profile based on structural responses (e.g., choosing "Sushi" combined with "Arcade"). The landing frame displays the structural output: *"I'll handle the plans; you just bring the sexy,"* wrapped within a self-assembling floating field of non-clickable, lightweight thematic vectors (such as animated chopsticks or neon joysticks) that align visually with her path selection.

### ### 1.3 Encrypted Shadow Admin Deck & Social Proof Scarcity Core

To preserve complete operational privacy, the admin workspace is hidden within plain sight. Triple-tapping the core profile illustration on the primary landing frame triggers a hidden gesture event sequence listener. If completed within a tight 1.2-second envelope, a localized canvas layer overlays a biometric WebAuthn prompt or security input matrix.

Once authenticated, the user gains entry to a high-density analytics dashboard tracking raw metrics, lead conversion lists, and the Dynamic Social Proof Scarcity Engine. This engine empowers the admin to explicitly inject and simulate active sessions, which triggers subtle user notifications stating that other regional invitation windows are currently filling up.

### ### 1.4 Biometric Intent & Velocity Telemetry

During the user journey, a background script actively captures and records continuous event logs without interrupting the layout frame rate. It evaluates variables including total session duration, total count of evasion triggers, touch approach velocity angles, and time spent observing the initial greeting layout. Upon successfully surrendering to the "Yes" selection path and finalizing scheduling parameters, the engine displays a humorous summary breakdown of their behavioral resistance metric (e.g., "Nice try—you attempted to evade me 14 times before finally choosing a date with KJ.\*").

---

## ## 2. Asynchronous Production Prompts

Feed these prompts into exactly 10 separate chat threads concurrently to build out your modules simultaneously:

### ### 🗨️ Chat 1: Database Schema Engine & Prisma ORM Layer

\* \*\*Target Files:\*\* `prisma/schema.prisma`, `lib/db.ts`

\* \*\*Prompt:\*\*

> Act as a Principal Database Architect. Create a highly optimized, clean database configuration structure utilizing Prisma for a Next.js 15 application. The core schema model needs to track `Lead` entities including: unique id string, timestamps, basic user variables (name, email, phone number), explicit multi-step selections (food preferences, activity environments, custom open answers), date/time selections, and an unstructured JSON record block dedicated to tracking interaction biometric events. Build a secondary model called `SystemConfig` to record global passcode configurations and current session simulation counters. Export a globally accessible, cached Prisma singleton instance within the `lib/db.ts` pipeline to avoid active connection footprint issues during rapid Hot Module

Replacement cycles in local dev modes. Do not write frontend components or API route implementations.

### 🗨️ Chat 2: Central API Routing Matrix & Security Validation Middleware

\* \*\*Target Files:\*\* ` /app/api/funnel/submit/route.ts`, ` /middleware.ts`

\* \*\*Prompt:\*\*

> Act as a Lead Backend Engineer. Write the main server-side Next.js route file to handle incoming JSON lead data payloads via `POST`. The input request payload must match raw text input values alongside complex behavioral tracking arrays. Provide structured try/catch parsing loops, write validation verification routines, and interface cleanly with a mock data structure stub mirroring the database layer. In addition, construct a secure, lightweight validation filter inside ` /middleware.ts` configured to check incoming header strings or authorization cookies when clients attempt to retrieve dashboard records. If credentials fail validation criteria, instantly yield a clean `401 Unauthorized` response payload. Do not alter UI layouts or affect schema structures.

### 🗨️ Chat 3: AI Aesthetic Match Scripting Engine & 120-Scenario Bank

\* \*\*Target Files:\*\* ` /lib/ai/scenarioEngine.ts`, ` /app/api/ai/generate-plan/route.ts`

\* \*\*Prompt:\*\*

> Act as an Algorithmic Logic & Scripting Systems Architect. Build an interactive client-side automation algorithm designed to select from 120 potential real-world scenario outputs based on user preferences. Design a clean structural dictionary that accepts preference metrics (e.g., "sushi" or "pasta", combined with "arcade" or "movies") and returns a premium date manifest structure. This structural response matrix must return: a personalized date summary title, a playful agenda timeline narrative, and an array of 6 to 8 matching vector asset keys to drive decorative UI rendering states. Provide robust, clean fallback profiles for edge conditions or empty options. Keep code completely separated from the UI visualization blocks.

### 🗨️ Chat 4: Continuous Interaction Telemetry & Velocity Tracking Core

\* \*\*Target Files:\*\* ` /hooks/useBiometricTracker.ts`, ` /context/

TelemetryContext.tsx`

\* \*\*Prompt:\*\*

> Act as a Biometric Interaction Data Engineer. Design a highly responsive, high-performance telemetry tracking state provider using React Context for a Next.js 15 app. The core context wrapper must capture mouse tracking paths, cursor velocities, and tap/touch movement vectors across various devices. Implement tracking hooks to log: cumulative time in active view, precise evasion attempts against protected layout elements, and coordinate maps. Ensure tracking hooks use optimized debounced update methods or passive event listener configurations to completely prevent browser layout stuttering or interface rendering degradation. Provide a clean, structured data extraction method called

``getTelemetryPayload()`` that transforms tracked metrics into a structured JSON payload for database upload.

### 🗨️ Chat 5: The Kinetic Landing Canvas & Evasive "No" Button Component

\* \*\*Target Files:\*\* ``/components/interactive/EvasiveButton.tsx``, ``/app/page.tsx``

\* \*\*Prompt:\*\*

> Act as a UI Physics Engineer & Designer. Build the main landing viewport layout following a clean, premium minimalist white aesthetic. The layout features an elegant, centered portrait frame showcasing the provided profile illustration of KJ, a prominent header message stating "Do you want to go on a date with KJ?", and two core interactive action options: "Yes" and "No". Build the evasive interaction physics within the "No" button component utilizing Framer Motion animations. Implement cursor and touch tracking hooks. When the proximity distance drops below 75px, recalculate coordinates to move the button away from the tracking vector. Build internal window boundary guards to keep the button fully within view via bounce reflections. Ensure the "Yes" action cleanly invokes a structural callback state to advance the user journey.

### 🗨️ Chat 6: Seductive Questionnaire Component Suite & Transition Layouts

\* \*\*Target Files:\*\* ``/components/funnel/QuestionnaireSuite.tsx``, ``/components/funnel/OptionCard.tsx``

\* \*\*Prompt:\*\*

> Act as an Interaction Design Specialist. Build an elegant, animated sequence containing three consecutive multi-option select cards that appear after the user clicks the primary "Yes" action. Design the interface using a minimalist white look with subtle drop-shadow accents. The selection loop covers: Card 1 ("Sushi" vs. "Pasta"), Card 2 ("Arcade" vs. "Movies"), and Card 3 (a playful qualifying prompt). Manage internal view step states with Framer Motion slide-in animations. Ensure each option click saves metrics into a local state repository and triggers smooth transitions to the next panel view without causing full page reloads.

### 🗨️ Chat 7: Minimalist Premium Micro-Calendar Scheduling Interface

\* \*\*Target Files:\*\* ``/components/funnel/DatePickerGrid.tsx``, ``/components/funnel/TimeSlotSelector.tsx``

\* \*\*Prompt:\*\*

> Act as a Client Workflow Developer. Build a clean, premium scheduling calendar grid optimized for mobile and desktop screens. Avoid using bulky external UI plugins; construct a raw grid layout styled with minimalist Tailwind utility classes. The interface renders a clean layout for choosing available calendar days alongside an elegant selector for booking times. Prevent multi-selection overlaps by implementing clean validation controls, and make sure that once a day and time are chosen, the selection triggers a callback hook containing a standardized Javascript Date object.

### ### 🗣️ Chat 8: Identity Vault, Data Validation Contact Form Layer

\* \*\*Target Files:\*\* ` /components/funnel/LeadCaptureForm.tsx `

\* \*\*Prompt:\*\*

> Act as a Form Logic & Client Validation Engineer. Design a minimalist, ultra-polished text entry form layer to collect user identity details before displaying the final results screen. The system must capture: full name, email address, and a contact phone number. Implement strict, clean client-side validation logic (e.g., regex checks for valid email structures and phone number lengths). Keep the visual styling fully aligned with the premium white aesthetic, utilizing ultra-clean text borders and subtle focus highlights. When form fields pass validation, pass data records outward via an `onSubmit()` callback.

### ### 🗣️ Chat 9: Visual Canvas Finalizer & Humorous Surrender Summary Screen

\* \*\*Target Files:\*\* ` /components/funnel/FinalCanvas.tsx ` , ` /components/funnel/SurrenderSummary.tsx `

\* \*\*Prompt:\*\*

> Act as a High-Fidelity Presentation Engineer. Build the final success confirmation screen and tracking summary view. The layout features the core message: `"I'll handle the plans; you just bring the sexy."` Set up an animated background field populated with 6 to 8 floating icon vectors generated by the script engine to reflect their choices. Below this, display a humorous telemetry summary based on data fetched from the tracking hook (e.g., `"Interaction analysis complete: You tried to click 'No' 12 times before surrendering to this date."`). Ensure all floating vectors are completely non-clickable to keep focus on the confirmation message.

### ### Chat 10: Encrypted Shadow Admin Interface & Gesture Matrix Capture

\* \*\*Target Files:\*\* ` /components/admin/GestureTrigger.tsx ` , ` /app/hidden-deck/page.tsx `

\* \*\*Prompt:\*\*

> Act as a Full-Stack Security & Dashboard Engineer. Build the hidden gesture listener matrix and private admin view. Create an invisible tap counter component on the main image element that tracks sequential interactions. When a triple-tap occurs within 1.2 seconds, display a password authentication overlay. Upon successful authentication, render a clean, high-density dashboard grid that lists collected lead contacts, preference choices, and interaction telemetry logs. Include a management section for the Scarcity Engine where the user can adjust active simulation counters to control user-facing notification systems.

## ## 5. Chat 11: Identity Overlay & Master Orchestration Schema

### ### 5.1 Dynamic Identity Injection Manifest

- \*\*Target File:\*\* ` /config/brandIdentity.ts `

- **Scope:** Define global content strings, imagery vectors, and specific scenario variants to overwrite generic layout blocks post-compilation.

```
``typescript
export const KJ_IDENTITY_MANIFEST = {
  profile: {
    name: "KJ",
    avatarUrl: "/assets/kj-smiling-portrait.jpg",
    bioHook: "I'll handle the plans; you just bring the sexy."
  },
  designSystem: {
    theme: "Strict Premium Minimalist White",
    accentGlow: "rgba(0, 0, 0, 0.03)",
    animationCurve: [0.16, 1, 0.3, 1] // Custom ease-out expo
  },
  curatedOptions: {
    food: {
      optionA: { label: "High-Fidelity Omakase", value: "sushi" },
      optionB: { label: "Artisan Lounge Pasta", value: "pasta" }
    },
    vibe: {
      optionA: { label: "Competitive Vector Arcade", value: "arcade" },
      optionB: { label: "Cinematic Late-Night Crawl", value: "movies" }
    }
  }
};
```

## 🚀 Chat Instance 11: Production Activation Prompt When your first 10 chats complete their parallel generation cycles, open your 11th chat container and hand it this precise instruction: > **[ROLE]:** Master Brand Orchestrator & Identity Systems Engineer. > > **[TARGET WORKSPACE]:** ``/config/brandIdentity.ts`, `/content/scenarios.json``, and UI configuration injects. > > **[TASK DIRECTIVE]:** You are responsible for executing the final styling and content pass across the entire web app engine. Your goal is to completely personalize the generic frontend slots to reflect a premium, genius, and highly curated personal brand for **KJ**. > > 1. Overwrite the baseline questionnaire layouts to utilize sophisticated, seductive option labels (e.g., swapping generic food/vibe options for high-end terms like "High-Fidelity Omakase", "Artisan Lounge Pasta", and "Cinematic Late-Night Crawls"). > 2. Inject a custom content array into the 120-scenario dictionary that builds premium, unforgettable date itineraries blending architectural style, creative lifestyle aesthetics, and high-performance, optimized dating frameworks. > 3. Ensure the initial white-themed hero interface perfectly highlights the smiling portrait element with the text, **"Do you want to go on a date with KJ?"**, keeping the code completely clean, decoupled, and configured as an elegant personal

lookbook presentation. Use local configuration objects to cleanly map variables without breaking the core backend routes built by parallel agents. This completes your 11-stage automaton matrix. Drop this into your development loop, and watch the entire engine align seamlessly with your exact creative frequency.